

ForgeWeb: Architecture-First Full-Stack Web Application Generation

Pratyush Manas, Rajeev Kumar, Sachin Yash Raj, Sanaa Kumari
Department of Computer Science and Engineering
BMS Institute of Technology and Management, Bengaluru, India
1BY23CS163 1BY23CS171 1BY23CS192 1BY23CS202

Abstract—ForgeWeb translates product descriptions into reviewed requirements and an approved architecture before coordinating source generation. The revised design retains Gemini for planning and React output and proposes a locally fine-tuned, pretrained LLaMA-based model for Python FastAPI generation. The workspace exposes files, previews, versions, and requirement links. Fourteen prior platform tests and type checking support the existing workflow, not the proposed model. Training data, checkpoint selection, and code-quality evaluation remain to be finalized. This paper separates implemented artifact management from proposed model adaptation and identifies execution-based testing as the next acceptance boundary.

Index Terms—AI-assisted development, web application generation, local language models, fine-tuning, requirement traceability.

I. INTRODUCTION

Turning a customer brief into software requires decisions about screens, data, permissions, and interactions. The result must represent the requested product: a restaurant needs a booking journey rather than a renamed dashboard, and an explicit color preference must affect the rendered interface.

Recent research connects language models to requirements specification, architectural reasoning, and code generation. ReqInOne addresses requirements-document generation [1], while requirements-to-code work examines the transition toward implementation [2]. Empirical SRS evaluation also makes requirements quality a research concern [3]. Building an application introduces further coordination across screens, routes, and files, with opportunities for users to correct the intended design before generation.

ForgeWeb connects a prompt, an approved specification, generated artifacts, and recorded checks. The revised backend design adapts a pretrained LLaMA-based model for local code generation; it does not claim a newly pretrained foundation model. The platform’s own interface remains separate from the customer’s requested design. Local backend inference does not make the whole pipeline offline because planning and frontend generation still use Gemini.

II. PROBLEM STATEMENT AND OBJECTIVES

Product descriptions combine features, preferences, and unstated assumptions. Generation may change a name while retaining an unsuitable layout, and an attractive preview may conceal missing server behavior. The problem is to preserve product intent while exposing enough evidence to assess the

result. ForgeWeb creates an architecture file before implementation and supports revision, approval, source inspection, and version history. Its objectives are to respect explicit design choices, coordinate frontend and backend contracts, and separate artifact creation from runtime correctness.

III. RELATED WORK

A. Requirements and Architectural Decisions

The requirements studies [1]–[3] motivate making the specification inspectable before implementation. Architectural-decision generation [4] and research on LLM assistance in architecture design [5] address a related stage: choosing and documenting design decisions. ForgeWeb uses an architecture document as an approval boundary, without claiming to be the first system to combine requirements and code generation.

B. Agents and Human Participation

SALLMA studies architecture for LLM-based multi-agent systems [6], while SOEN-101 connects code generation to software-process roles [7]. Human-in-the-loop development agents [8] provide a relevant comparison for user participation. Design and refactoring frameworks [9], multi-agent code-generation studies [10], and retrieval/communication-oriented agent architectures [11] broaden this setting. ForgeWeb contributes a particular workflow integration; its present task records should not be confused with independently reasoning agents.

C. Local Generation and Traceability

Research on locally deployed open-source models [12] motivates evaluating the proposed local backend generator under actual hardware constraints. CodeWisp examines retrieval-guided code generation and completion [13], suggesting a future source-context mechanism. Neither establishes which checkpoint ForgeWeb should adopt.

Architecture-component extraction [14], LiSSA [15], Code Gradients [16], and security-requirement trace recovery [17] address different ways to connect software artifacts. ForgeWeb instead records requirement-to-file associations during generation. Those associations aid inspection but do not establish semantic satisfaction or security enforcement.

D. Validation and Research Position

Assured LLM-based engineering [18], static-analysis-assisted review [19], and interactive code-generation evaluation [20] motivate feedback beyond text generation. Accuracy/quality/performance evaluation [21], holistic evaluation [22], and a code-generation review [23] support assessing several quality dimensions. Unit-test generation [24] and industrial acceptance-test generation [25] are relevant to future executable checks. ForgeWeb’s contribution is the integration of planning, approval, generation, and project history; superiority over these approaches requires comparative experiments and is not claimed here.

IV. SYSTEM ARCHITECTURE

A. User Layer and Control Service

The user layer uses React and TypeScript. It contains the prompt composer, proposal review, project drawer, file viewer, preview, and version controls. A Node.js HTTP service coordinates these operations, with Vite middleware exposing the same handler during development. Workflow states distinguish planning, confirmation, generation, review, validation, and completion or failure. Figure 1 outlines this separation.

Project state is persisted in a local JSON store. Queued writes and temporary-file replacement reduce conflicts within the local process. This arrangement supports the prototype, but is not a substitute for a transactional multi-tenant database. Database information shown in the workspace describes intended application entities; it does not prove that a database has been provisioned.

B. Generation and Artifact Storage

The proposed split pipeline uses Gemini for planning and frontend generation and a locally served, fine-tuned LLaMA-based model for backend source. A local inference adapter would receive the approved architecture, entities, routes, and API contract, then return a validated file manifest. Backend generation would no longer depend on a hosted backend-code API. The local checkpoint, model size, hardware, and serving configuration are not yet fixed; no latency or privacy improvement is claimed without measurement.

Local generators cover selected commerce, restaurant, portal, inventory, scheduling, and generic-site cases. They provide a bounded fallback rather than unlimited design originality. Generator mode and failure messages remain attached to the result. Stored artifacts include source, a shared contract, documentation, preview HTML, and file digests. Snapshots support inspection, restoration, and ZIP export.

C. Proposed Model Adaptation

Training would pair architecture and API specifications with reviewed Python implementations. Data must be license-compatible, stripped of secrets, deduplicated, and split by project to reduce train-test leakage. A pretrained LLaMA-based checkpoint would be fine-tuned on these pairs and served locally after evaluation. The untouched base model and adapted model should be compared on held-out builds, API

behavior, and authorization tests. Checkpoint identity, dataset provenance, training settings, and hardware must be recorded before reporting results.

V. METHODOLOGY

A. Planning and Approval

The planner turns a validated prompt into a proposed specification and `ARCHITECTURE.md`. Explicit names and visual preferences should survive normalization. The user can edit the request or confirm the proposal; implementation begins only after confirmation. This boundary checks agreement about scope, not the correctness of later code. Figure 2 shows the sequence and failure branch.

B. Frontend and Backend Coordination

The approved specification guides React source, styles, FastAPI routes, schemas, access-control files, and supporting tests. A shared JSON contract expresses the intended route and data agreement. Figure 3 separates these files from the preview path. Contract presence reduces ambiguity but cannot establish that implementations agree: a booking form and its endpoint can still interpret dates or identifiers differently.

The embedded preview runs stored HTML in an iframe with script and form permissions. It does not start the generated Python service. Local preview interactions must therefore be distinguished from verified frontend-to-backend requests. Running the exported application requires its dependencies and data services to be configured.

C. Review and Traceability

Planning, implementation, review, and validation records describe objectives, allowed paths, and acceptance criteria. Project policy draws on the supplied Karpathy-derived engineering guidance and gstack workflow material: clarify assumptions, prefer a sufficient solution, limit changes to scope, and inspect evidence. These engineering repositories are not counted as research publications.

The current checks examine paths, manifests, expected artifacts, requirement mappings, and provenance. The internal graph links projects, requirements, tasks, files, and checks. A requirement-to-file edge records an association, not proof that a feature works. Similarly, the graph is not an externally parsed call graph from Code Review Graph. Stronger acceptance would combine the current structural checks with clean builds, API requests, and browser interaction tests against the generated application.

VI. EVALUATION AND DISCUSSION

The earlier platform evaluation passed fourteen top-level tests and TypeScript checking. Table I reports that baseline, not a rerun of a local-model integration. Temporary stores and deterministic or injected responses exercise orchestration; no fine-tuned checkpoint was tested. Local-model accuracy, runtime, and resource use remain unmeasured.

Approval checks enforce confirmation before implementation. Domain checks examine product-specific output and explicit colors, but not perceived uniqueness. Workspace checks

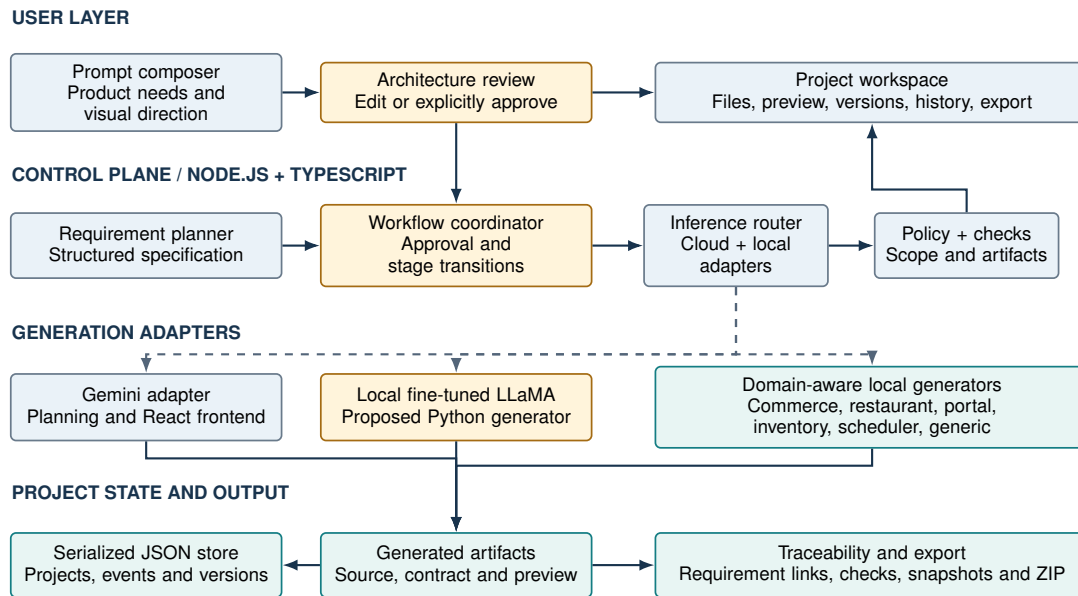


Fig. 1. Revised system architecture: the existing control plane with a proposed locally fine-tuned LLaMA backend generator. Gemini remains responsible for planning and frontend output. The trained checkpoint is not yet evaluated.

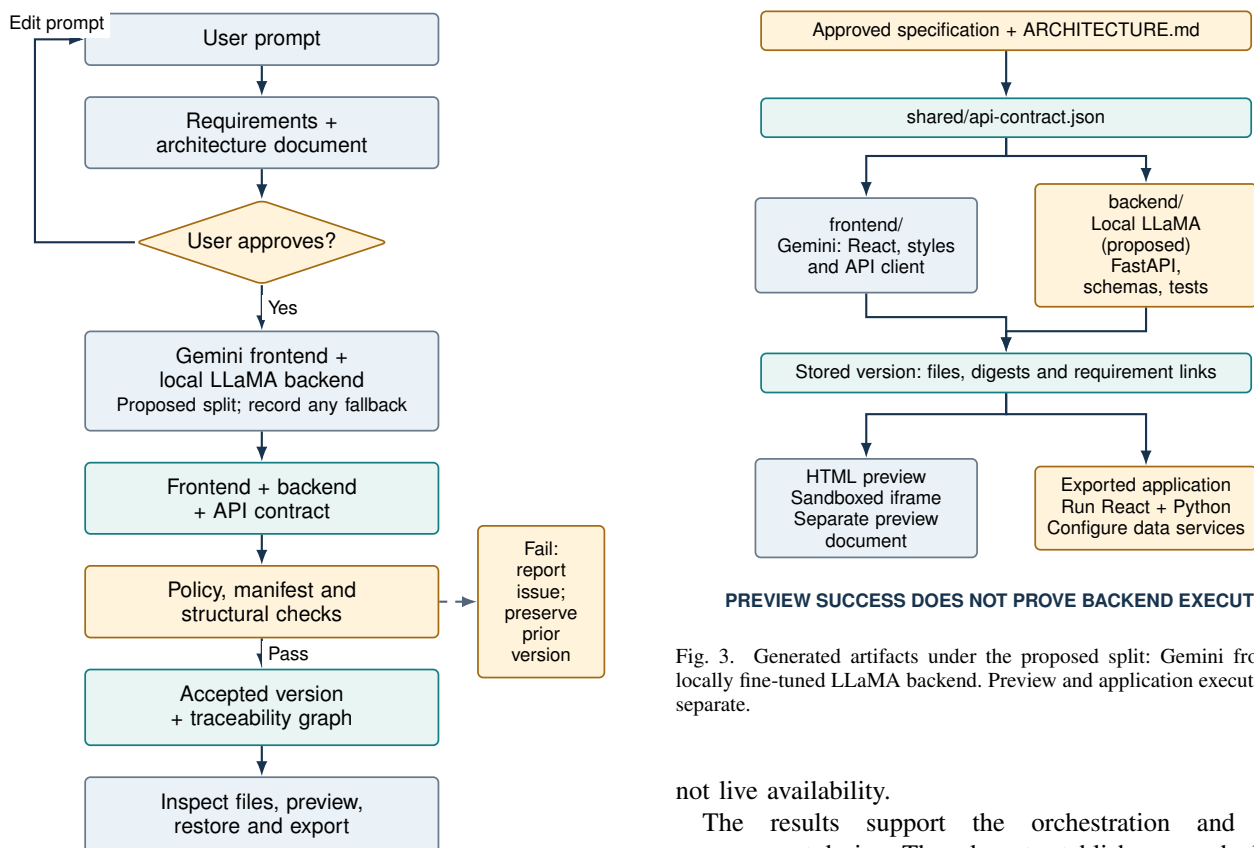


Fig. 2. Revised data flow with proposed local-model backend generation. A recorded template fallback is distinct from model inference.

Fig. 3. Generated artifacts under the proposed split: Gemini frontend and locally fine-tuned LLaMA backend. Preview and application execution remain separate.

cover snapshots, restoration, persistence, export, and safe failure. Provider fixtures verify plan and manifest handling,

not live availability.

The results support the orchestration and artifact-management design. They do not establish secure deployment, payment correctness, or successful execution of every generated interaction. A useful next experiment would compare direct generation with architecture-first generation on the same prompts and provider settings. Measurements should include requirement-level task success, contract agreement, clean-build success, visual adherence, latency, and cost. Such comparative

TABLE I
PRIOR PLATFORM CHECKS (NOT LOCAL-MODEL RESULTS)

Test group	Cases	Result
HTTP and approval lifecycle	2	Passed
Domain and visual instructions	4	Passed
Provider fixture and selection	2	Passed
Plan normalization	1	Passed
Workspace, versions, compatibility	3	Passed
Prompt and path rejection	2	Passed
Total	14	Passed
Platform type checking	—	Passed

measurements have not been conducted here.

VII. LIMITATIONS AND FUTURE WORK

Source checks can miss broken handlers, incorrect permissions, and mismatched payloads. Fallback designs remain limited, model responses vary, and local persistence is unsuitable for production-scale tenancy.

The immediate priority is selecting, fine-tuning, and evaluating the local LLaMA-based checkpoint, then connecting its manifest output to the workflow. Isolated execution, persistent application data, and browser tests remain necessary. Training does not itself establish code correctness or security, and human assessment must verify visual requirements.

VIII. CONCLUSION

ForgeWeb makes planning and approval explicit before source generation. The revised design combines Gemini-assisted frontend generation with a proposed locally finetuned LLaMA backend model. Existing checks support the surrounding workflow, not the untested checkpoint. Training evidence and executed application tests are needed before assessing the revised pipeline’s effectiveness.

REFERENCES

- [1] T. Zhu, L. C. Cordeiro, and Y. Sun, “ReqInOne: A Large Language Model-Based Agent for Software Requirements Specification Generation,” in *Proc. IEEE RE*, 2025, pp. 449–457, doi: 10.1109/re63999.2025.00054.
- [2] B. Wei, “Requirements are All You Need: From Requirements to Code with LLMs,” in *Proc. IEEE RE*, 2024, pp. 416–422, doi: 10.1109/re59067.2024.00049.
- [3] M. Krishna, B. Gaur, A. Verma, and P. Jalote, “Using LLMs in Software Requirements Specifications: An Empirical Evaluation,” in *Proc. IEEE RE*, 2024, pp. 475–483, doi: 10.1109/re59067.2024.00056.
- [4] R. Dhar, K. Vaidhyanathan, and V. Varma, “Can LLMs Generate Architectural Design Decisions? - An Exploratory Empirical Study,” in *Proc. IEEE ICSE*, 2024, pp. 79–89, doi: 10.1109/icsa59870.2024.00016.
- [5] H. Cervantes, Y. Cai, and R. Kazman, “Large Language Models as Assistants in Software Architecture Design,” *IEEE Software*, vol. 43, no. 4, pp. 87–95, 2026, doi: 10.1109/ms.2026.3663353.
- [6] M. Becattini, R. Verdecchia, and E. Vicario, “SALLMA: A Software Architecture for LLM-Based Multi-Agent Systems,” in *Proc. IEEE/ACM SATrends*, 2025, pp. 5–8, doi: 10.1109/satrends66715.2025.00006.
- [7] F. Lin, D. J. Kim, and T.-H. Chen, “SOEN-101: Code Generation by Emulating Software Process Models Using Large Language Model Agents,” in *Proc. IEEE/ACM ICSE*, 2025, pp. 1527–1539, doi: 10.1109/icse55347.2025.00140.
- [8] W. Takerngsaksiri, J. Pasuksmit, P. Thongtanunam *et al.*, “Human-In-The-Loop Software Development Agents,” in *Proc. IEEE/ACM ICSE-SEIP*, 2025, pp. 342–352, doi: 10.1109/icse-seip66354.2025.00036.
- [9] V. Rajendran, D. Besiahgari, S. C. Patil, M. Chandrashekaraiiah, and V. Challagulla, “A Multi-Agent LLM Environment for Software Design and Refactoring: A Conceptual Framework,” in *Proc. SoutheastCon*, 2025, pp. 488–493, doi: 10.1109/southeastcon56624.2025.10971563.
- [10] R. Ramírez-Rueda, E. Benítez-Guerrero, C. Mezura-Godoy, and E. Bárcenas, “Transforming Software Development: A Study on the Integration of Multi-Agent Systems and Large Language Models for Automatic Code Generation,” in *Proc. CONISOF*, 2024, pp. 11–20, doi: 10.1109/conisof63288.2024.00013.
- [11] V. Gogineni, “LLM-Powered Multi-Agent Systems: A Technical Framework for Collaborative Intelligence Through Optimized Knowledge Retrieval and Communication,” in *Proc. AIRC*, 2025, pp. 452–456, doi: 10.1109/airc64931.2025.11077480.
- [12] T. Kechaoui, M. W. Ouhab, B. Djamaa, and M. R. Senouci, “Locally-deployed Open-source LLMs for Code Generation: Promises and Challenges,” in *Proc. PAIS*, 2025, pp. 1–6, doi: 10.1109/pais66004.2025.11126523.
- [13] H. El Atrassi, Y. El Idrissi, and Y. Benkaouz, “CodeWisp: AST Guided Retrieval Augmented Generation for Code Generation and Completion,” in *Proc. WINCOM*, 2025, pp. 1–6, doi: 10.1109/wincom65874.2025.11313399.
- [14] D. FuchSS, H. Liu, T. Hey, J. Keim, and A. Koziolek, “Enabling Architecture Traceability by LLM-based Architecture Component Name Extraction,” in *Proc. IEEE ICSE*, 2025, pp. 1–12, doi: 10.1109/icsa65012.2025.00011.
- [15] D. FuchSS, T. Hey, J. Keim *et al.*, “LiSSA: Toward Generic Traceability Link Recovery Through Retrieval-Augmented Generation,” in *Proc. IEEE/ACM ICSE*, 2025, pp. 1396–1408, doi: 10.1109/icse55347.2025.00186.
- [16] M. North, A. Atapour-Abarghouei, and N. Bencomo, “Code Gradients: Towards Automated Traceability of LLM-Generated Code,” in *Proc. IEEE RE*, 2024, pp. 321–329, doi: 10.1109/re59067.2024.00038.
- [17] J.-M. PaSSlack, A. Specht, M. Herrmann *et al.*, “Can we use LLMs to recover Trace Links between Source Code and Security Requirements?” in *Proc. IEEE REW*, 2025, pp. 223–232, doi: 10.1109/rew66121.2025.00038.
- [18] N. Alshahwan, M. Harman, I. Harper, A. Marginean, S. Sengupta, and E. Wang, “Assured LLM-Based Software Engineering,” in *Proc. IEEE/ACM IntenSE*, 2024, pp. 7–12. [Online]. Available: <https://ieeexplore.ieee.org/document/10669832>
- [19] I. Jaoua, O. B. Sghaier, and H. Sahraoui, “Combining Large Language Models with Static Analyzers for Code Review Generation,” in *Proc. IEEE/ACM MSR*, 2025, pp. 174–186, doi: 10.1109/msr66628.2025.00038.
- [20] D. Shaikhelislamov, M. Drobyshevskiy, and A. Belevantsev, “LLM-based Interactive Code Generation: Empirical Evaluation,” in *Proc. Ivanikov ISPRAS*, 2024, pp. 1–5, doi: 10.1109/ispras64596.2024.10899123.
- [21] M. A. Shehab, M. Wardat, S. Omari, and Y. Jararweh, “Evaluating Large Language Models for Code Generation: Assessing Accuracy, Quality, and Performance,” in *Proc. FLLM*, 2024, pp. 407–416, doi: 10.1109/film63129.2024.10852439.
- [22] J. Höll and J. Kufer, “Holistic evaluation of LLM-Based Code Generation,” in *Proc. FLLM*, 2025, pp. 234–241, doi: 10.1109/film67465.2025.11390916.
- [23] J. Wang and Y. Chen, “A Review on Code Generation with LLMs: Application and Evaluation,” in *Proc. IEEE MedAI*, 2023, pp. 284–289, doi: 10.1109/medai59581.2023.00044.
- [24] M. Schäfer, S. Nadi, A. Eghbali, and F. Tip, “An Empirical Evaluation of Using Large Language Models for Automated Unit Test Generation,” *IEEE Transactions on Software Engineering*, vol. 50, no. 1, pp. 85–105, 2024, doi: 10.1109/tse.2023.3334955.
- [25] M. Ferreira, L. Viegas, J. P. Faria, and B. Lima, “Acceptance Test Generation with Large Language Models: An Industrial Case Study,” in *Proc. IEEE/ACM AST*, 2025, pp. 1–11, doi: 10.1109/ast66626.2025.00007.